



Hochschule für Technik,
Wirtschaft und Kultur Leipzig

FAKULTÄT INGENIEURWISSENSCHAFTEN

PROJEKT SEIKO DRUCKER

ELEKTRO UND INFORMATIONSTECHNIK (KOOPERATIV)

Belegarbeit

Autoren

Jonathan Blänsdorf, Paul Freiberger

Hochschul-Betreuer

Henning Nagel, Ingo Reinhold

Leipzig, 22. September 2026

Inhaltsverzeichnis

1	Einführung	1
2	Ausgangssituation	2
2.1	Aktuellen Ist-Zustand beschreiben	2
2.2	Problem oder Bedarf klar darstellen	2
2.3	Rahmenbedingungen (technisch / organisatorisch) nennen	2
3	Aufgabenstellung	3
3.1	Konkrete Aufgaben und Ziele formulieren	3
3.2	Abgrenzung: Was ist Teil der Arbeit, was nicht	3
4	Projektablauf	4
4.1	Vorgehensweise / Methodik beschreiben	4
4.1.1	Projektentscheidung und erste Sichtung	4
4.1.2	Automatisierungsmethode suchen	4
4.1.3	Machbarkeit erproben	4
4.1.4	Entscheidung für Automatisierungslösung	5
4.1.5	Durchführung	5
4.1.6	Testen und Fehlerbehebung	6
4.1.7	Dokumentation und Vorstellung	6
4.2	Eingesetzte Programme	7
5	Lösung	9
5.1	Spalten Programm	9
5.2	Erweiterungsmodule und Parameter	10
5.3	Checkliste	10
5.4	Hilfsfunktionen aufstellen	11
5.5	Schrittfunktion Definition	13
5.6	Startabfragen	19
6	Bedienung	20
6.1	Hardware Vorbereitung	20
6.2	Software Vorbereitung	20
7	Zusammenfassung	21
7.1	Ausblick	21

Abbildungsverzeichnis	22
8 Anhang	23
Quellenverzeichnis	28

1 Einführung

Im Zuge der fortschreitenden Digitalisierung nahezu aller Lebensbereiche könnte der Eindruck entstehen, dass die physische Drucktechnik ihren Zenit überschritten hat und auf dem Weg zur Bedeutungslosigkeit ist. Dieser Eindruck täuscht jedoch. Gerade weil digitale Systeme immer leistungsfähiger werden, wächst gleichzeitig der Bedarf an präzisen, materialvielfältigen und funktionalen Druckverfahren. Ob in der Elektronikfertigung, der Medizintechnik, der Verpackungsindustrie oder der Forschung: Die physische Drucktechnik findet heutzutage in zahlreichen Bereichen Anwendung.

Die vorliegende Belegarbeit entstand im Rahmen eines studentischen Projekts. Ziel dieser Arbeit ist es, die Entwicklung und Umsetzung eines automatisierten Systems zu dokumentieren, mit dem verschiedene Drucktechniken praktisch erprobt und miteinander verglichen werden können. Mithilfe dieser Vorrichtung erhielt die Gruppe von Studierenden die Möglichkeit, eigene Ansätze zur Automatisierung einer industrienahen Maschine zu entwickeln und umzusetzen. Dieser handlungsorientierte Ansatz ermöglicht nicht nur den Erwerb technischen Fachwissens, sondern schärft zugleich das Verständnis für die Zusammenhänge zwischen Antriebstechnik, softwarebasierten Lösungsansätzen und industrieller Anwendung.

Die Relevanz des Projekts ergibt sich aus mehreren Gesichtspunkten. Zum einen wurde sich mit verschiedenen Antriebstechniken auseinandergesetzt, wodurch die im Studium erworbenen Kenntnisse in einer praxisnahen Anwendung eingesetzt werden konnten. Zum anderen mussten aufgrund der bereits vorhandenen technischen Rahmenbedingungen kreative, softwarebasierte Lösungsansätze entwickelt werden, die verschiedene Bereiche des Studiums miteinander verbinden. Darüber hinaus förderte die projektbezogene Zusammenarbeit eine klare und konstruktive Kommunikation, die wesentlich zum Erfolg des Projekts beitrug.

Die Arbeit ist wie folgt aufgebaut: Zunächst werden die technischen Grundlagen sowie die im Projekt eingesetzten Werkzeuge und Methoden erläutert. Anschließend werden verschiedene Ansätze zur Umsetzung der Automatisierung betrachtet und hinsichtlich ihrer Eignung für das Projekt bewertet. Darauf aufbauend werden der technische Aufbau des Systems, die entwickelte Steuerungssoftware und der automatisierte Ablauf beschrieben. Im weiteren Verlauf werden die Durchführung und die Ergebnisse des Projekts dargestellt und analysiert. Abschließend werden die gewonnenen Erkenntnisse zusammengefasst und mögliche Weiterentwicklungen des Systems aufgezeigt.

2 Ausgangssituation

2.1 Aktuellen Ist-Zustand beschreiben

Am Anfang des Projekts wurde der Drucker bereits vollständig und betriebsbereit übergeben. Zudem wurde ein Laptop mitgeliefert, mit welchem die beiden Linearantriebe für x- und y-Achse angesteuert werden konnten. Diese Ansteuerung erfolgte allerdings nur händisch im Hand- bzw. Tippbetrieb. Somit befand der Drucker in einem nicht automatisierten Zustand.

2.2 Problem oder Bedarf klar darstellen

Um zukünftigen Studierenden die Funktionalität des Druckers erklären zu können und verschiedene Druckköpfe an diesem auszutesten, soll dieser in einem automatisierten Zustand ablaufen. Dabei sollen die Studierenden nicht zwingend vor ein einzelnes fertiges Programm gestellt werden, sondern sollen sie auch selber Abläufewiederholungen erstellen können, die für ihre jeweiligen Druckköpfe angepasst werden sollen.

2.3 Rahmenbedingungen (technisch / organisatorisch) nennen

Dem Drucker beigelegt war eine Dokumentation und eine Betriebsanleitung. Die Dokumentation war allerdings auf Japanisch verfasst, was eine Herausforderung in der Inbetriebnahme des Druckers darstellt, weil Japanisch aufgrund der anderen Schriftzeichen und der vielfältigen Interpretationsmöglichkeiten schwierig zu übersetzen ist. Der Drucker wurde vollständig aufgebaut vorgefunden und mit einem Laptop PC geliefert welcher vorinstalliert mit Windows 8 Betriebssystem und der für die Antriebs-Steuerung benötigten PCon und MR Configurator Software.

Die Organisatorischen Rahmenbedingungen wie zeitliche Planung und Kostenmöglichkeiten wurden von den Hochschulbetreuern vorgegeben. Dabei wurde klargestellt, dass finanziell nur auf Restbestände von Laboren oder anderen Einrichtungen zurückgegriffen werden konnte. Das Projekt wurde planungsmäßig auf das Wintersemester 25/26 und Sommersemester 26 ausgelegt. Das Projekt sollte keinen sonderlichen Zeitdruck hatte, allerdings mussten die Studierenden spätestens Ende des Sommersemesters 2026 dieses Projekt abschließen, weil in diesem Zeitraum das Studium der Studierenden zu Ende gehen würde.

3 Aufgabenstellung

3.1 Konkrete Aufgaben und Ziele formulieren

Aus der Kommunikation mit den Betreuenden wurde folgende Aufgabenstellung definiert:

Die Steuerung soll automatisiert ablaufen anstatt einen Bediener die Abläufe vollständig ausführen zu lassen. Die Nachrüstung soll möglichst kostengünstig mit dem bestehenden Drucker und dem zugehörigen Laptop ausgeführt werden. Die Abläufe sollen für verschiedene Drucktechniken individuell angepasst werden können. Eine Anleitung für die Arbeit mit der Automatisierungslösung soll mit einer übersichtlichen Projektdokumentation zusammen bereitgestellt werden. Das Projekt soll den Hochschulbetreuern präsentiert und samt der Doku vollständig übergeben werden.

3.2 Abgrenzung: Was ist Teil der Arbeit, was nicht

Erwartete Ergebnisse benennen

Das System soll den Druckkopf-Träger nach Vollendung steuern und über die Druckfläche führen. Die Bedienung soll sich grundlegend auf die Auswahl und Ausführung von Programmen beziehen und nicht das Ausführen einzelner Schrittschritte. Die verfügbaren Ressourcen bei der Bearbeitung beschränken sich auf die verfügbaren Bauteile des Automatisierungslabors der HTWK Leipzig, Neubeschaffungen sind in dem Budget nicht vorgesehen.

4 Projektablauf

4.1 Vorgehensweise / Methodik beschreiben

Der Ablauf des Projektes hat sich Thematisch in 7 verschiedene Etappen eingliedern lassen und wurden in der geschilderten Reihenfolge ausgeführt.

4.1.1 Projektentscheidung und erste Sichtung

Um ein tieferes Verständnis über die Funktion des bestehenden Druckers zu erlangen wird die mitgelieferte Dokumentation gelesen. Dazu liegt auch eine Anleitung zu Inbetriebnahme der grundlegenden Funktion und Steuerung bei.

Daraufhin stand die Definition der Aufgabenstellung an. Die Inbetriebnahme des Druckers wurde zusammen mit den Betreuern der Hochschule durchgeführt um die Ausgangssituation festzustellen.

4.1.2 Automatisierungsmethode suchen

Ein unmittelbar ersichtliches Problem war, dass die Dokumentation des Druckers auf Japanisch verfasst wurde und nur einzelne Textpassagen in dem Benutzerhandbuch auf Englisch übersetzt wurden.

Es wurden im Bezug auf die Rahmenbedingungen mögliche Lösungsansätze aufgestellt die Automatisierung des Systems auf Ebene der Steuerung über der vorhandenen Feldebene (Linearantrieben und Steuerantriebe) anzusetzen. Der erste aufgestellte Ansatz war die Umsetzung der Steuerung mithilfe einer speicherprogrammierbaren Steuerung (SPS). SPS werden in der industriellen Automatisierung zur Steuerung von Maschinen und technischen Prozessen eingesetzt und sind in der Normenreihe IEC 61131 beschrieben [1]. Aufgrund ihrer Programmierbarkeit, Anpassungsfähigkeit und Eignung für die industrielle Maschinen- und Prozesssteuerung haben sie sich seit mehreren Jahrzehnten als wesentliche Automatisierungskomponente etabliert [2]. Eine weitere Überlegung war den Austausch der Datenpakete von Laptop zu den Steuerungseinheiten der Antriebe zu analysieren und nachgebildet über einen Mikrocontroller an die Steuerungseinheiten den Ablauf zu bestimmen. Die letzte Methode orientierte sich an dem bereits etablierten Ablauf, wobei die Bedienung der grafischen Nutzeroberfläche von einem Programm nachgestellt werden kann.

4.1.3 Machbarkeit erproben

Für die Umsetzung der Steuerung mittels einer SPS war eine Siemens ET200-SP zur Verfügung gestellt wurden. Dieses Modell der SPS wird auch typischer Weise in der Industrie eingesetzt für die Ansteuerung von Antrieben. Die zwei vorhandenen Antriebe zusammen mit den vorhandenen Steuerungseinheiten/Encoder basieren jedoch nicht auf den gleichen Kommunikationsprotokollen wie die SPS es anbietet. Für die Umsetzung mit der vorhandenen SPS wäre ein Übersetzungsmodul notwendig um die Kommunikation zu errichten. Die Notwendigkeit von einer Neuanschaffung und die damit einhergehende Übersteigerung des Budget lässt diese Lösung ausscheiden, würde in der Umsetzung sehr Aufwendig sein aber auch am dichtesten am Industrie Standard sein.

Um die bisherige Kommunikation, zwischen Laptop und der Linearantrieb zu replizieren scheint es möglich, die gesendeten Protokolle über Positionierungs Daten von Steuerung und Antrieb austauschen, einfach über die USB Schnittstelle abzufassen, analysieren und als nachgestelltes Protokoll zu schicken und die Positionierung der Antriebe zu steuern. Die Software auf dem Laptop die für beide Antriebe die Kommunikation übernimmt, sendet und empfängt verschlüsselte Pakete über die Position der Antriebe. Die Verschlüsselten Pakete lassen sich nicht ohne weiteres entschlüsseln und beinhalten einen Heartbeat welcher von dem Kommunikationspartner aufrechterhalten werden

muss. Schlussendlich scheitert diese Methode daran dass sich in der Industrie die Informationssicherheit gezielt gegen Fremdeinwirkung und Sabotage richtet.

Da sich die bisherigen Ansätze zwischen der Feldebene und Steuerungsebene wenig Erfolg bereitet haben hat sich der dritte Ansatz bewiesen. Dieser basiert auf der Annahme dass der Ausgangszustand in dem der Drucker schon alle relevanten Funktionen aufweist die Automatisierung bisher schon mit der richtigen Eingabe innerhalb der beiden Antriebssoftwares ausführbar war. Die Ebene zwischen Mensch und Maschine wird typischer Weise nicht als Möglichkeit weiter zu Automatisieren, bietet jedoch in unserem Fall mit den gegebenen Rahmenbedingungen die Möglichkeit Steuerung nach einem bestimmten Ablauf zu konzipieren. Um einen Menschlichen Bediener mittels Software nachzubilden lohnt sich der Einsatz von Methoden die bereits auf genau solche Anforderungen konzipiert sind und im gegebenen Rahmen des Projektes Platz finden. Um die Funktionen die vorher der Mensch übernommen hat ist für die Automatisierung ein Teil Computer Vision nötig welcher in der Software die nötigen Kennzeichen erkennen kann. Für das Betätigen von Interaktiven Feldern innerhalb der Antriebssoftware können die Eingabe Signal von Peripherie Elementen simuliert werden.

4.1.4 Entscheidung für Automatisierungslösung

Die Lösung muss im Laufe der Bearbeitung auch mit den Hochschulischen Betreuern abgeklärt werden. Die Lösungsansätze der Implementierung einer SPS, sowie die Nachbildung des Kommunikationsprotokolls übersteigen die vorgegebene Rahmenbedingung im Sinne des Budgets/ Ressourcen da beide Möglichkeiten zwar möglich sein, jedoch die Neuanschaffung von Übersetzungsmodulen verlangt. Der Ansatz der Nachbildung von Eingangs und Ausgangssignalen mittels Computer Vision und simulierten Menschlichen Eingaben verspricht die besten Aussichten auf die Automatisierung der Drucker. Die grafische Benutzeroberfläche dient als Grundlage des Ist-Zustands des Drucker Systems bietet bereits eine funktionstüchtige Basis. Mit beiden Hochschul-Betreuern wurde sich geeinigt dass die Einhaltung der Rahmenbedingungen über den Anforderungen von Industriellen Standards steht und eine experimentelle sowie auch individuelle Lösung die Vorstellungen an das Projekt näher treffen kann. Für die Automatisierung wird eine Programmierung in Python vorgesehen sowie einige Erweiterungsmodule die in Python implementiert werden können um so weitere Möglichkeiten der Computer Vision, Ablaufprogrammierung oder Eingaben Simulation zu gewähren.

4.1.5 Durchführung

Erstellung von zwei Programmen mit denen über wenige eingaben der Ablauf gesteuert werden kann. Mittels der Dokumentation von Python sowie pyautogui lassen sich die Programmierfortschritt schnell erreichen. Es wurden für den gesamten Ablauf zwei verschiedene Programme mit python erstellt.

1. Pfadplanung: SpaltenPosition.py
Für das erste Programm war vorhergesehen eine Möglichkeit zu schaffen verschiedene Abläufe über den Druckbereich. Es wird interaktive gestaltet sodass keine Änderung von Parametern im Programmcode erforderlich ist sondern bei der Ausführung abgefragt wird welche Systemwerte vorhanden sind und daraufhin wird die Datei erstellt die in der Antriebssoftware den Verlauf ändert.
2. Ablaufsteuerung: DruckerProgram(getestet).py
Dieses Programm startet die Bedienung der Antriebssteuerung und fragt zu Beginn einige Startbedingungen ab um eine Fehlerfreien Grundzustand vorrauszusetzen. Die auszuführenden Schritte sollen jeweils durch eine Bedingung von einander getrennt sein. Das können Zeitintervalle sein aber auch das Abwarten von bestimmten visuellen Anzeichen über den Zustand des Druckers.

4.1.6 Testen und Fehlerbehebung

Die entstandenen Programme zur Steuerung der Antriebe sollen auch wiederholbar ablaufen können und auch sicher gegen zufällige Fehler gestaltet sein. Um solche Fehler zu vermeiden wurde das Programm nach Erreichen der nötigen Funktion erweitert an die Anforderungen an Benutzerfreundlichkeit sowie Benutzersicherheit.

4.1.7 Dokumentation und Vorstellung

Die Dokumentation und Präsentation des Projekts dienen dazu, den Entwicklungsprozess, die eingesetzten Methoden und die erzielten Ergebnisse nachvollziehbar darzustellen. Dadurch können die einzelnen Arbeitsschritte, aufgetretenen Herausforderungen und erarbeiteten Lösungen auch von anderen Personen verstanden und bewertet werden. Die Belegarbeit bietet zudem eine Grundlage für spätere Erweiterungen und Verbesserungen des Systems. Die Präsentation ergänzt die schriftliche Ausarbeitung, indem die Funktionsweise und die Bedeutung des Projekts anschaulich vermittelt werden. Dadurch wird sowohl der technische Erkenntnisgewinn als auch die praktische Relevanz der entwickelten Automatisierung aufgezeigt.

4.2 Eingesetzte Programme

Für die Umsetzung und Untersuchung des Druckerprojekts wurden verschiedene Softwareprogramme eingesetzt, die jeweils unterschiedliche Aufgaben innerhalb des Projekts übernehmen. Durch das Zusammenspiel dieser Programme können sowohl die technische Steuerung als auch die softwaregestützte Automatisierung und Untersuchung des Drucksystems umgesetzt werden.

1. Advanced USB Port Monitor: Analyse der USB-Kommunikation

Der Advanced USB Port Monitor der AGG Software wird zur Beobachtung und Analyse der Kommunikation zwischen dem Computer und den angeschlossenen Geräten eingesetzt. Die Software kann USB-Datenverkehr erfassen, anzeigen und auswerten. Dadurch lassen sich unter anderem übertragene Daten, Geräteparameter und mögliche Kommunikationsfehler untersuchen [3].

Im Druckerprojekt ermöglicht der USB Port Monitor die Analyse der Schnittstelle zwischen der Steuerungssoftware und dem Drucksystem.

2. Python: Ablaufsteuerung und Automatisierung

Python ist eine interpretierte, universell einsetzbare Programmiersprache, die sich durch eine vergleichsweise klare Syntax und eine große Anzahl verfügbarer Bibliotheken auszeichnet [4]. Im vorliegenden Projekt wird Python als übergeordnete Ablaufsteuerung eingesetzt. Das Programm koordiniert die einzelnen Arbeitsschritte, überprüft Ausgangsbedingungen und steuert die Reihenfolge der Bedienhandlungen.

Python übernimmt damit eine vermittelnde Funktion zwischen der Bedienoberfläche der verwendeten Software und dem technischen Druckprozess. Die einzelnen Schritte des Ablaufs können in Funktionen strukturiert und bei Bedarf wiederholt oder erweitert werden. Basierend auf dem Windows 8 Betriebssystem muss die dafür geeignete Version von Python Version 3.8.10 genutzt werden.

3. Python-Module `time` und `sys`

Die Standardbibliothek von Python stellt mit dem Modul `time` Funktionen für Zeitmessungen und zeitgesteuerte Wartezeiten bereit [5]. Im Projekt wird insbesondere `time.sleep()` verwendet, um nach einzelnen Bedienhandlungen definierte Wartezeiten einzuhalten. Diese Wartezeiten sind erforderlich, damit die gesteuerten Anwendungen ausreichend Zeit für Fensterwechsel, Positionsänderungen und die Verarbeitung von Befehlen haben.

Das Modul `sys` stellt Funktionen und Variablen bereit, die mit der Python-Laufzeitumgebung interagieren [6]. Im Programm wird `sys.exit()` verwendet, um die Automatisierung bei einem nicht behebbaren Fehler kontrolliert abubrechen.

4. PyAutoGUI: Automatisierung der grafischen Benutzeroberfläche

PyAutoGUI ist eine Python-Bibliothek zur Steuerung von Maus und Tastatur. Sie ermöglicht unter anderem das Bewegen des Mauszeigers, das Ausführen von Mausklicks sowie die Simulation von Tastatureingaben [7]. Zusätzlich können Bildschirmausschnitte erfasst und vorgegebene Bildvorlagen auf dem Bildschirm gesucht werden.

Im vorliegenden Projekt wird PyAutoGUI verwendet, um die Bedienung der verwendeten PC-Interface-, pCon- und MR-Config-Software zu automatisieren. Da für die proprietären Anwendungen keine geeignete direkte Programmierschnittstelle zur Verfügung steht, erfolgt die Steuerung auf der Ebene der grafischen Benutzeroberfläche. Dazu sucht das Programm zuvor erstellte PNG-Bildvorlagen, beispielsweise für Schaltflächen, Fenster und Taskleistensymbole, und führt anschließend die entsprechenden Mausklicks aus.

Diese Vorgehensweise ist der Robotic Process Automation auf GUI-Ebene zuzuordnen. Sie ermöglicht eine Automatisierung, ohne den Quellcode der verwendeten Herstellerprogramme verändern zu müssen. Die Zuverlässigkeit hängt dabei unter anderem von der Bildschirmauflösung, der Skalierung und der gleichbleibenden Darstellung der Benutzeroberfläche ab.

5. NumPy: Numerische Berechnungen

NumPy ist eine weit verbreitete Python-Bibliothek für numerische Berechnungen und die Verarbeitung mehrdimensionaler Datenstrukturen [8]. Die Bibliothek stellt unter anderem effiziente Arrays und mathematische Funktionen bereit.

NumPy kann im weiteren Verlauf des Projekts eingesetzt werden, wenn beispielsweise Positionsdaten, Koordinatentransformationen, Bewegungsprofile oder Messwerte der Druckversuche numerisch verarbeitet werden müssen.

6. Thonny: Entwicklungsumgebung

Thonny ist eine integrierte Entwicklungsumgebung für Python, die insbesondere für einen übersichtlichen Einstieg in die Programmierung entwickelt wurde [9]. Die Entwicklungsumgebung bietet unter anderem einen integrierten Python-Interpreter, eine interaktive Konsole und Debugging-Funktionen.

Im Projekt wird Thonny zur Erstellung, Ausführung und Fehlersuche des Python-Programms verwendet. Das auf dem Laptop befindliche Windows 8 Betriebssystem sowie Python Version 3.8.10 erfordert einen geeigneten Editor, der mit diesen veralteten Versionen funktionieren kann.

5 Lösung

Die nach dem Testen und den Fehlerbehebungen entstandene Programme sind hier in Funktion erklärt.

5.1 Spalten Programm

```
1 from datetime import datetime
2
3 # Anzahl der Spalten abfragen
4 spalten = int(input("Anzahl der Spalten: "))
5
6 # Dateiname
7 dateiname = "Tabelle.ptpa"
8
9 # Aktuelle Zeit f r Header
10 jetzt = datetime.now()
11
12 with open(dateiname, "w", encoding="utf-8") as f:
13
14     # Header
15     f.write(f";DATE={jetzt.strftime('%Y/%m/%d')}\n")
16     f.write(f";TIME={jetzt.strftime('%H:%M:%S')}\n")
17     f.write(";CTL=AE30000F\n")
18     f.write(";AXIS=00\n")
19     f.write(";PC=0D000300\n")
20     f.write(";POSITION=00000300\n")
21     f.write(";LENDEC_POS=00000002\n")
22     f.write(";ACHS=00000000\n")
23     f.write(";AEHS=00000000\n")
24     f.write(";ARSF=00000000\n")
25     f.write(";FVER=0000000C\n")
26     f.write(";PRG1=FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF\n")
27     f.write(";PRG2=FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF\n")
28     f.write(";STP=0000000000000000\n")
29     f.write(";RPT=00000000\n")
30
31     # Zeile 0
32     f.write(
33         "0,0.00,150.00,0.30,0.30,0,0,0.10,0.00,0.00,00000000,00,\n"
34     )
35
36     # Weitere Zeilen abh ngig von der Spaltenanzahl
37     x_wert = 300.0
38
39     for i in range(1, spalten + 1):
40         f.write(
41             f"{i},{x_wert:.2f}\n"
42             f";50.00,0.30,0.30,0,0,0.10,0.00,0.00,00000000,00,\n"
43         )
44         x_wert += 50.0
45
46 print(f"Datei '{dateiname}' wurde erfolgreich erstellt.")
```

Listing 1: Erweiterungsmodule und Programmparameter

Hier das Programm beschreiben.

5.2 Erweiterungsmodule und Parameter

```
1 import pyautogui
2 import time
3 import sys
4
5 # =====
6 # KONFIGURATION      Zeitlimits und Wartezeiten (in Sekunden)
7 # =====
8 CONFIG = {
9     "min_wait_pcon_step":      3.0,    # Mindestlaufzeit nach pcon-Schritt
10    "max_wait_pcon_step":      10.0,    # Maximallaufzeit f r pcon-Schritt
11    "min_wait_mrconfig_step":  3.0,    # Mindestlaufzeit nach MR Config-
12    "max_wait_switch_window":  10.0,    # Maximallaufzeit f r
13    "wait_after_opstart":      12.0,    # Wartezeit nach opstartmrconfig
14    "wait_after_arrow":        3.0,    # Wartezeit nach Pfeil-Klick
15    "search_confidence":       0.8,    # Erkennungssicherheit f r
16    "pyautogui"
17 }
```

Listing 2: Erweiterungsmodule und Programmparameter

Die zu importierenden Pakete werden aufgerufen und die Parameter die innerhalb des Programmes benutzt werden, werden initialisiert. Diese Werte können hier angepasst werden, wenn die Abfolge der Schritte mit anderen Zeitparametern ablaufen sollen.

5.3 Checkliste

```
18 # =====
19 # CHECKLISTE
20 # =====
21 CHECKLIST = [
22     "Stecker eingesteckt",
23     "Handbetrieb eingestellt",
24     "MR Config gestartet",
25     "Positionsdatei in MR Config geladen",
26     "pCon Software gestartet",
27     "Positionsdatei in pCon geladen",
28 ]
```

Listing 3: Liste der Startbedingungen

Hier wird die Liste der Bedingungen festgehalten, die vor dem Start abgehakt werden müssen um das Programm aus einem gesicherten Zustand anfangen zu können.

5.4 Hilfsfunktionen aufstellen

```
30 # =====
31 # HILFSFUNKTIONEN
32 # =====
33
34 def find_and_click(image_path, timeout=10.0, min_wait=0.0, description=""):
35     """
36     Sucht ein Bild auf dem Bildschirm und klickt darauf.
37     Wartet mindestens min_wait Sekunden und maximal timeout Sekunden.
38     Gibt True zur ck wenn erfolgreich, False wenn nicht gefunden.
39     """
40     start_time = time.time()
41     print(f"        Suche nach: {description or image_path}")
42
43     while time.time() - start_time < timeout:
44         try:
45             location = pyautogui.locateCenterOnScreen(
46                 image_path,
47                 confidence=CONFIG["search_confidence"]
48             )
49             if location:
50                 elapsed = time.time() - start_time
51                 # Mindestlaufzeit einhalten
52                 if elapsed < min_wait:
53                     time.sleep(min_wait - elapsed)
54                 pyautogui.click(location)
55                 print(f"        Gefunden und geklickt: {description or
56                     image_path}")
57                 return True
58             except pyautogui.ImageNotFoundException:
59                 pass
60                 time.sleep(0.5)
61
62     print(f"        FEHLER: '{description or image_path}' nicht gefunden
63         innerhalb von {timeout}s!")
64     return False
```

Listing 4: Funktion die die Screenshots findet

Die Funktion von autogui die innerhalb des Programmes auf die vorab gewählten Bildschirmaufnahmen zugreift, diese innerhalb des derzeitigen Desktops sucht und mit dem Mauszeiger anklickt.

```
64 def is_on_screen(image_path):
65     """
66     Pr ft ob ein Bild auf dem Bildschirm sichtbar ist (ohne zu klicken).
67     """
68     try:
69         location = pyautogui.locateOnScreen(
70             image_path,
71             confidence=CONFIG["search_confidence"]
72         )
73         return location is not None
74     except pyautogui.ImageNotFoundException:
75         return False
```

Listing 5: Funktion die die Anzeige prüft

Hierbei handelt es sich um den Programmausschnitt in dem die Funktion von autogui beschrieben wird die auf dem Bildschirm die passende .png Datei sucht um zu kontrollieren ob sie derzeit auf dem Bildschirm abgebildet ist.

```

77 def ensure_pcon_active():
78     """
79     Stellt sicher, dass das pCon-Fenster aktiv ist.
80     """
81     print("\n          Pr fe ob pCon-Fenster aktiv ist...")
82     if is_on_screen("currentwindowpcon.png"):
83         print("          pCon-Fenster ist bereits aktiv.")
84         return True
85     else:
86         print("          pCon nicht aktiv      suche Icon in Taskleiste...")
87         success = find_and_click(
88             "pconicon.png",
89             timeout=CONFIG["max_wait_switch_window"],
90             description="pCon Taskleisten-Icon"
91         )
92         if not success:
93             print("          KRITISCH: pCon konnte nicht aktiviert werden!")
94             sys.exit(1)
95             time.sleep(1.0)
96             return True

```

Listing 6: Aktives PCon Fenster prüfen

In dieser Funktion wird geprüft ob das benötigte Fenster der PCon Applikation geöffnet ist.

```

98 def ensure_mrconfig_active():
99     """
100     Stellt sicher, dass das MR Config-Fenster aktiv ist.
101     """
102     print("\n          Wechsle zu MR Config...")
103     if is_on_screen("currentwindowmrconfig.png"):
104         print("          MR Config-Fenster ist bereits aktiv.")
105         return True
106     else:
107         print("          MR Config nicht aktiv      suche Icon in Taskleiste
108             ...")
109         success = find_and_click(
110             "mrconfigicon.png",
111             timeout=CONFIG["max_wait_switch_window"],
112             description="MR Config Taskleisten-Icon"
113         )
114         if not success:
115             print("          KRITISCH: MR Config konnte nicht aktiviert werden!")
116             sys.exit(1)
117             time.sleep(1.0)
118             return True

```

Listing 7: Aktives MR Configurator Fenster prüfen

In dieser Funktion wird geprüft ob das benötigte Fenster des MR-Configurator Applikation geöffnet ist.

5.5 Schrittfunktion Definition

```
119 # =====
120 # SCHRITT-FUNKTIONEN
121 # =====
122
123 def mrconfig_sequence():
124     """
125     SchrittMRConfig      Die immer gleiche MR Config-Sequenz.
126     Wiederverwendbar in SchrittPcon0 und SchrittPcon.
127     Alle Pfeil- und OP-Start-Schritte haben Mindestlaufzeiten.
128     """
129     print("\n          Starte MR Config Sequenz...")
130
131     if not is_on_screen("currentwindowmrconfig.png"):
132         print("          MR Config Fenster nicht erkannt!")
133         sys.exit(1)
134
135     if not is_on_screen("pointtableno1.png"):
136         print("          Punkttabelle Nr. 1 nicht gefunden!")
137         sys.exit(1)
138
139     #          Schritt 1: OP Start
140
141     if not find_and_click(
142         "opstartmrconfig.png",
143         timeout=CONFIG["max_wait_pcon_step"],
144         min_wait=CONFIG["min_wait_mrconfig_step"],
145         description="OP Start MR Config (1)"
146     ):
147         sys.exit(1)
148     time.sleep(CONFIG["wait_after_opstart"])
149
150     #          Schritt 2: Pfeil HOCH
151
152     if not find_and_click(
153         "pfeilhoch.png",
154         timeout=CONFIG["max_wait_pcon_step"],
155         min_wait=CONFIG["min_wait_mrconfig_step"],
156         description="Pfeil Hoch"
157     ):
158         sys.exit(1)
159     time.sleep(CONFIG["wait_after_arrow"])
```

Listing 8: Anfang der MR Configurator Schrittkette, MR Schritt 1 und 2

Die Schritte die der, durch den MR Configurator gesteuerte Linearantrieb, ausführen muss, sind identisch. Es müssen bei diesem Schritt nur 2 verschiedene Positionen angefahren werden, jedoch abhängig von der Spaltenanzahl mehrfach wiederholt werden.

```

159      #           Schritt 3: OP Start

160      if not find_and_click(
161          "opstartmrconfig.png",
162          timeout=CONFIG["max_wait_pcon_step"],
163          min_wait=CONFIG["min_wait_mrconfig_step"],
164          description="OP Start MR Config (2)"
165      ):
166          sys.exit(1)
167      time.sleep(CONFIG["wait_after_opstart"])
168
169      #           Schritt 4: Pfeil RUNTER

170      if not find_and_click(
171          "pfeilrunter.png",
172          timeout=CONFIG["max_wait_pcon_step"],
173          min_wait=CONFIG["min_wait_mrconfig_step"],
174          description="Pfeil Runter"
175      ):
176          sys.exit(1)
177      time.sleep(CONFIG["wait_after_arrow"])
178
179      #           Schritt 5: OP Start (letzter)

180      if not find_and_click(
181          "opstartmrconfig.png",
182          timeout=CONFIG["max_wait_pcon_step"],
183          min_wait=CONFIG["min_wait_mrconfig_step"],
184          description="OP Start MR Config (3)"
185      ):
186          sys.exit(1)
187      time.sleep(CONFIG["wait_after_opstart"])
188
189      print("           MR Config Sequenz abgeschlossen.")

```

Listing 9: MR Schritt 3,4 und 5

```

191 def schritt_pcon0():
192     """
193     SchrittPcon0      Wird NUR EINMAL am Anfang ausgef hrt.
194     Initialisierungsschritt: Erster Schritt + Erster Spaltenpunkt + MR
        Config.
195
196     Schritt 1 (Schritt in pCon ausw hlen) wird NUR ausgef hrt,
197     wenn pos0.png NICHT auf dem Bildschirm gefunden wird.
198     """
199     print(f"\n{'='*55}")
200     print(f"      SchrittPcon0      Initialisierung (einmalig)")
201     print(f"{'='*55}")
202
203     # 1. pCon aktivieren
204     ensure_pcon_active()
205
206     #      Schritt 1: NUR ausf hren wenn pos0.png NICHT sichtbar
207     print("\n      Pr fe ob Position 0 bereits aktiv (pos0.png)...")
208     if not find_and_click(
209         "schritt_pcon.png",
210         timeout=CONFIG["max_wait_pcon_step"],
211         min_wait=CONFIG["min_wait_pcon_step"],
212         description="Schritt in pCon ausw hlen (Schritt 1)"
213     ):
214         sys.exit(1)
215     else:
216         print("      pos0.png gefunden      Position bereits aktiv, Schritt
217             1 wird      bersprungen .")
218
219     #      Schritt 2: Ersten Spaltenpunkt anfahren (IMMER)
220
221     print("\n      Schritt 2: Ersten Spaltenpunkt anfahren...")
222     if not find_and_click(
223         "schritt_pcon.png",
224         timeout=CONFIG["max_wait_pcon_step"],
225         min_wait=CONFIG["min_wait_pcon_step"],
226         description="Erster Spaltenpunkt"
227     ):
228         sys.exit(1)
229
230     # Mindestlaufzeit vor Fensterwechsel
231     time.sleep(CONFIG["min_wait_pcon_step"])
232
233     #      Schritt 3: Zu MR Config wechseln
234
235     ensure_mrconfig_active()
236
237     #      Schritt 4: MR Config Sequenz ausf hren
238
239     mrconfig_sequence()
240
241     #      Schritt 5: Zur ck zu pCon
242
243     ensure_pcon_active()
244
245     print(f"\n      SchrittPcon0 abgeschlossen.")

```

Listing 10: Initialschritt unabhängig von der Spaltenanzahl

Der Initialschritt muss von der PCon Software ausgeführt werden um sicher zu stellen dass sich

der Druckkopfträger auf der vorgesehenen Startposition bereitsteht. Danach kann die Abfolge der Schritte beginnen.

```

242 def schritt_pcon(spalten_nummer):
243     """
244     SchrittPcon      Wird f r JEDE weitere Spalte wiederholt.
245     F hrt den n chsten Spaltenpunkt an und f hrt MR Config Sequenz aus.
246     """
247     print(f"\n{'='*55}")
248     print(f"      SchrittPcon      Spalte {spalten_nummer}")
249     print(f"{'='*55}")
250
251     # 1. pCon aktivieren
252     ensure_pcon_active()
253
254     # 2. N chsten Spaltenpunkt anfahren
255     print(f"\n      Spaltenpunkt {spalten_nummer} anfahren...")
256     if not find_and_click(
257         "schritt_pcon.png",
258         timeout=CONFIG["max_wait_pcon_step"],
259         min_wait=CONFIG["min_wait_pcon_step"],
260         description=f"Spaltenpunkt {spalten_nummer}"
261     ):
262         sys.exit(1)
263
264     # Mindestlaufzeit vor Fensterwechsel
265     time.sleep(CONFIG["min_wait_pcon_step"])
266
267     # 3. Zu MR Config wechseln
268     ensure_mrconfig_active()
269
270     # 4. MR Config Sequenz ausf hren
271     mrconfig_sequence()
272
273     # 5. Zur ck zu pCon
274     ensure_pcon_active()
275
276     print(f"\n      SchrittPcon f r Spalte {spalten_nummer} abgeschlossen.
277           ")

```

Listing 11: PCon Schrittkette abhängig von der Spaltenanzahl des Testversuches

Die für den Testversuch gewählte Aufteilung der Druckoberfläche in Spalten, bestimmt innerhalb des Programmes wie häufig die Schrittfolge wiederholt werden muss.

```

278 def schritt_pcon_abschluss():
279     """
280     SchrittPconAbschluss      Wird NUR EINMAL ganz am Ende ausgef hrt.
281     Wiederholt einen letzten pCon-Schritt nach der finalen MR Config
282     Sequenz.
283     """
284     print(f"\n{'='*55}")
285     print(f"      SchrittPconAbschluss      Letzter pCon-Schritt")
286     print(f"{'='*55}")
287
288     # 1. pCon aktivieren
289     ensure_pcon_active()
290
291     # 2. Letzten pCon-Schritt ausf hren
292     print("\n      Letzten pCon-Schritt ausf hren...")
293     if not find_and_click(
294         "schritt_pcon.png",
295         timeout=CONFIG["max_wait_pcon_step"],
296         min_wait=CONFIG["min_wait_pcon_step"],
297         description="Letzter pCon-Schritt (Abschluss)"
298     ):
299         sys.exit(1)
300
301     # Mindestlaufzeit nach letztem Schritt
302     time.sleep(CONFIG["min_wait_pcon_step"])
303
304     print(f"\n      Abschlussschritt in pCon erfolgreich ausgef hrt.")

```

Listing 12: PCon Schlussschritt unabhängig von Spaltenanzahl

Damit bei Wiederholungen der gesamten Schrittfolge die nötige Ausgangsstellung schneller erreicht werden kann, endet jede Schrittfolge damit den Druckkopfträger wieder an die Ausgangsposition zurückzuführen.

5.6 Startabfragen

```
305 # =====
306 # CHECKLISTE ABFRAGEN
307 # =====
308
309 def run_checklist():
310     """
311     Fragt die Ausgangsbedingungen ab.
312     Gibt erst frei wenn alle Punkte best tigt wurden.
313     Gibt die Anzahl der Spalten zur ck.
314     """
315     print("\n" + "="*55)
316     print("    SEIKO DRUCKER AUTOMATISIERUNG        STARTCHECKLISTE")
317     print("="*55)
318     print("Bitte alle Punkte pr fen und mit 'j' best tigen.\n")
319
320     for i, item in enumerate(CHECKLIST, start=1):
321         while True:
322             answer = input(f"    [{i}/{len(CHECKLIST)}] {item}        Bereit? (j
323                             /n): ").strip().lower()
324             if answer == 'j':
325                 print(f"        Best tigt: {item}\n")
326                 break
327             elif answer == 'n':
328                 print(f"        Bitte zuerst '{item}' erledigen und dann
329                             erneut best tigen.\n")
330             else:
331                 print("    Bitte 'j' f r Ja oder 'n' f r Nein eingeben.")
```

Listing 13: Checkliste zur Bestätigung aufrufen

Die Checkliste wird ausgegeben und muss händisch für jede Bedingung bestätigt werden bevor das Programm anlaufen kann.

```
331 # Anzahl Spalten abfragen
332 while True:
333     try:
334         anzahl = int(input("\n Wie viele Spalten sollen verarbeitet
335                             werden? "))
336         if anzahl > 0:
337             print(f"        Anzahl Spalten: {anzahl}\n")
338             break
339         else:
340             print("    Bitte eine positive Zahl eingeben.")
341     except ValueError:
342         print("    Bitte eine g ltige Zahl eingeben.")
343
344     print("\n n    Checkliste vollst ndig        Automatisierung startet!\n")
345     return anzahl
```

Listing 14: Anzahl der Spalten abfragen

Die für den Testvorgang bestimmte Spaltenanzahl wird hier in einer Eingabefenster abgefragt.

6 Bedienung

Um die Ablaufsteuerung des Druckers gesichert ausführen zu können sind vorab einige Handgriffe zu tätigen. Dabei wird unterschieden in die Vorbereitung von Hardware, womit die sichere Ausgangssituation der elektromechanischen Bestandteile gewährleistet wird.

6.1 Hardware Vorbereitung

Die Linearantriebe des Druckers händisch in die Ausgangspositionen bringen und danach elektrisch sicher zuschalten.

1. Eine Sichtprüfung des Innenraums (ggf. Fremdkörper entfernen).
2. Den CEE Stecker einstecken und Spannung anlegen.
3. Den Drehschalter auf Maintenance (Handbetrieb) schalten.
4. Den Starttaster (grün) betätigen.
5. Die beiden Linearantriebe sollten nun unter Spannung stehen und nehmen ihre Position auf.
6. Aus vorherigen Abläufen können die Linearantriebe auch die Druckplatte blockieren.
7. Um die Platte mit den Druckmedien zu bestücken können die Linearantriebe per Hand aus dem Weg geschoben werden. **Vorsicht Quetschgefahr!**
8. Das Druckmedium kann mit Betätigung des Tasters -P1 innen im Druckraum an die Druckplatte gesaugt werden
9. Druckraumtüren schließen (Quetschgefahr während Ablauf verhindern).
10. Aus dem Schaltschrank führen die Datenkabel der zwei Steuermodule in den USB-Hub.
11. Das USB-Kabel des Laptops wird mit dem USB-Hub verbunden.
12. Den Laptop starten und sich mit dem Benutzer `student` anmelden.

6.2 Software Vorbereitung

Die Ablaufsteuerung über den Laptop programmseitig vorbereiten und sicher ausführen.

1. Den Python Editor „thony“ starten.
2. Das Pythonprogramm „SpaltenPosition.py“ zur Erstellung des Spalten Ablaufes in Thonny ausführen und z.B. unter „`C:\Users\student\Desktop\3SpaltenAblauf.py`“ abspeichern.
3. Die Steuerapplikation „MR Configurator“ starten.
4. Die Positionierungsprogramm laden.
5. Die Steuerapplikation „MR Configurator“ starten.
6. Das erstellte Positionierungsprogramm von Schritt 2 aus dem Ablageort importieren.
7. In Thonny das Pythonprogramm „DruckerProgram(getestet)“ zur Ablaufsteuerung ausführen.
8. Die erscheinende Checkliste Schritt für Schritt ausführen und die gewählte Spaltenanzahl eingeben.
9. Danach sollte das Programm automatisch starten.
10. Bei fehlerfreiem Ablauf sollte der Ausgangszustand zu Beginn und Ende wiederhergestellt werden (Antriebe auf Position 0 stellen).

7 Zusammenfassung

Das Ziel eines Automatisierten Systems konnte während des Projektverlaufes nach Zieldefinition und Festlegung auf Rahmenbedingungen erreicht werden. Durch die Absprache mit den Hochschulbetreuern waren die Erwartungen an das Projekt auch transparent gestellt sodass Missverständnisse verhindert werden konnten. Die Funktion die der Drucker durch das Implementieren der Python Programme gewonnen hat ermöglichen nun die Nutzung des Druckers für verschiedene Testzwecke im Feld der Lern- und Testprozesse.

7.1 Ausblick

Basierend auf dieser Dokumentation ist es möglich, weitere Funktionen in den automatisierten Ablauf zu integrieren. Die Erläuterung der Python-Programme kann dabei als Grundlage für zukünftige Erweiterungen dienen. Denkbar wäre beispielsweise, den gesamten Ablauf abhängig von einer Benutzereingabe wiederholt auszuführen oder gezielt nur eine Teilmenge der Druckoberfläche abzufahren. Dadurch könnte der Drucker flexibler für unterschiedliche Testreihen und Versuchsaufbauten eingesetzt werden. Die Struktur des Programms ermöglicht es, Anpassungen schrittweise umzusetzen. Durch die vorliegende Dokumentation können die einzelnen Programmfunktionen nachvollzogen und notwendige Änderungen gezielt vorgenommen werden. Damit bildet das Projekt eine Grundlage für die weitere Nutzung und Optimierung des Drucksystems im Rahmen zukünftiger Lern- und Testprozesse.

Abbildungsverzeichnis

1	Gesamtansicht des Drucker Systems	23
2	Bauraum des Drucker Systems	23
3	Rückseite, Linearantrieb IAI (von PCon gesteuert)	24
4	Vorderseite, Linearantrieb IAI (von PCon gesteuert)	25
5	Rückseite, Linearantrieb von Mitsubishi (von MR Configurator gesteuert)	25
6	Ablauf bei 3 Spalten	26
7	Ablauf bei 5 Spalten	27

8 Anhang

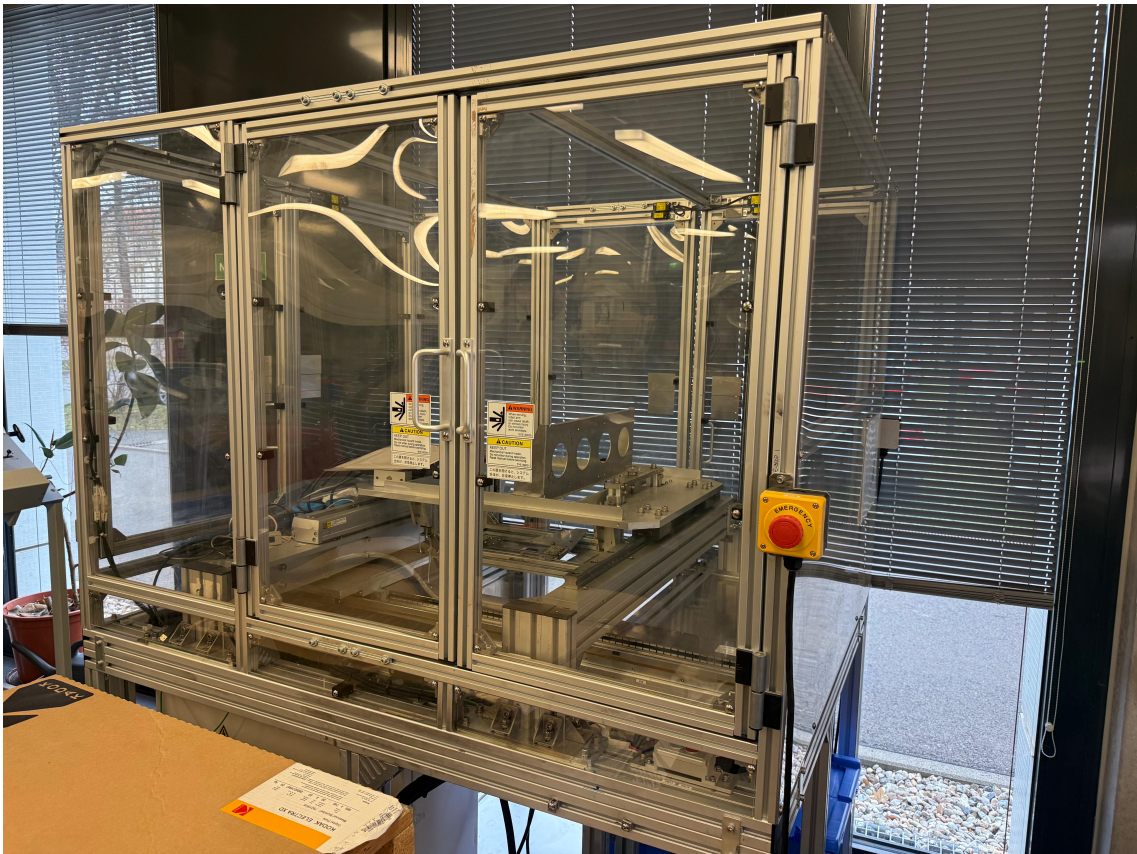


Abbildung 1: Gesamtansicht des Drucker Systems

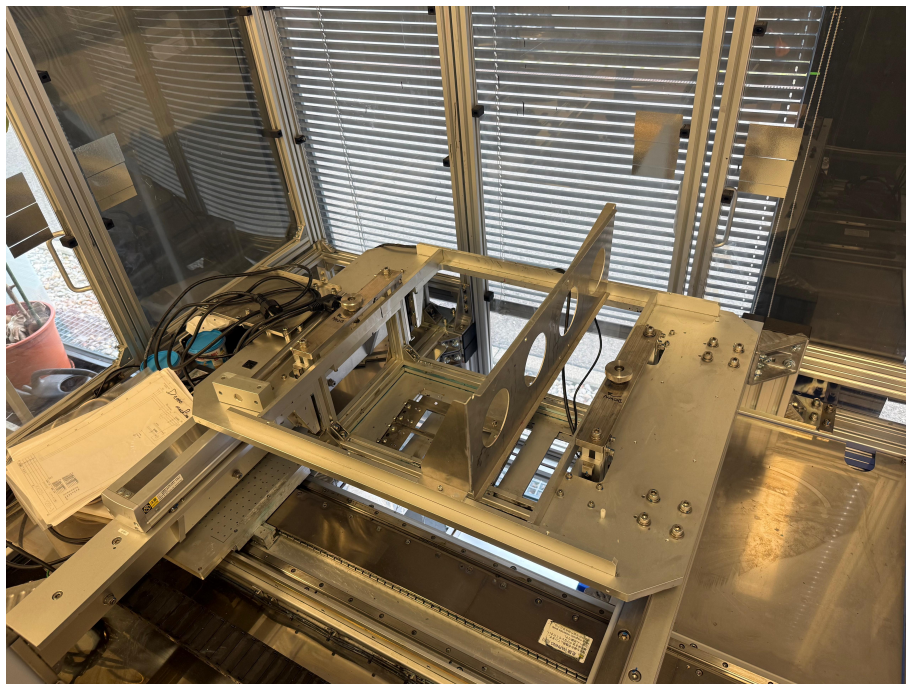


Abbildung 2: Bauraum des Drucker Systems



Abbildung 3: Rückseite, Linearantrieb IAI (von PCon gesteuert)



Abbildung 4: Vorderseite, Linearantrieb IAI (von PCon gesteuert)



Abbildung 5: Rückseite, Linearantrieb von Mitsubishi (von MR Configurator gesteuert)

3 Spalten

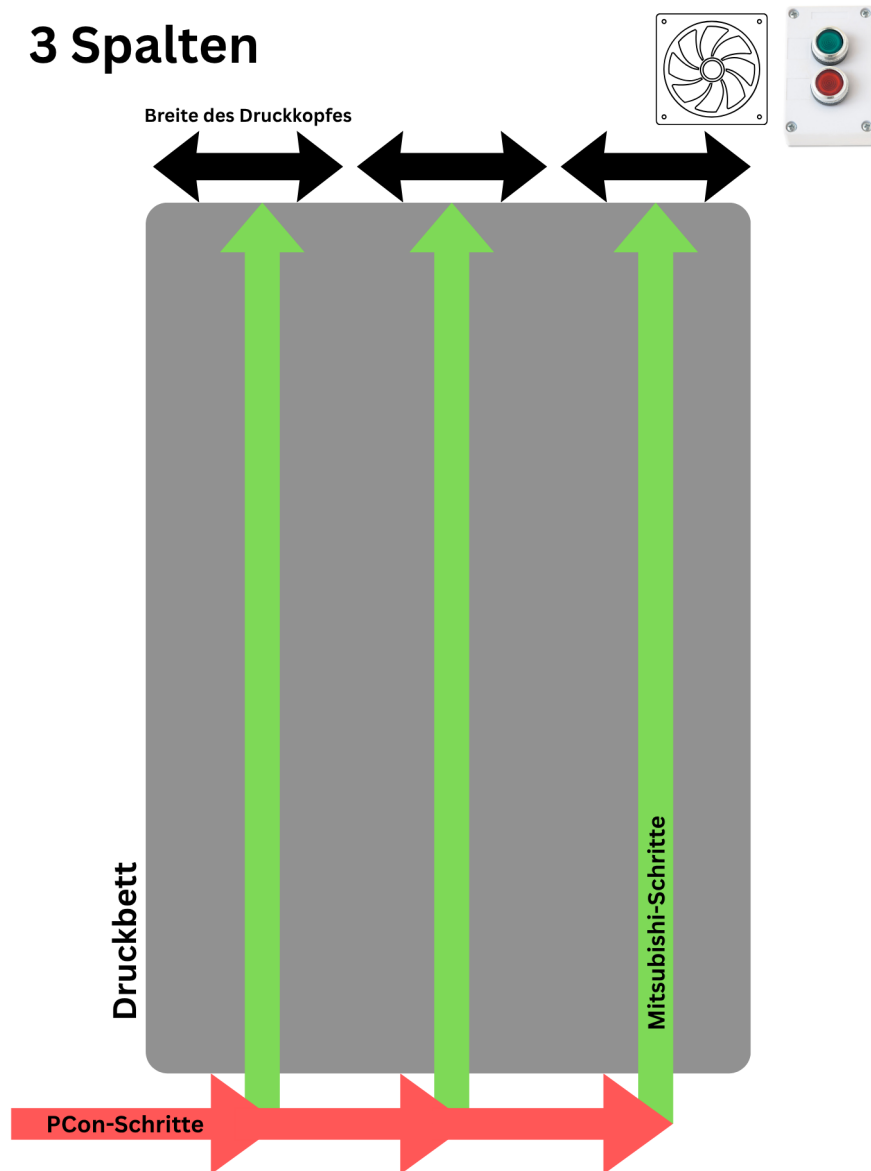


Abbildung 6: Ablauf bei 3 Spalten

5 Spalten

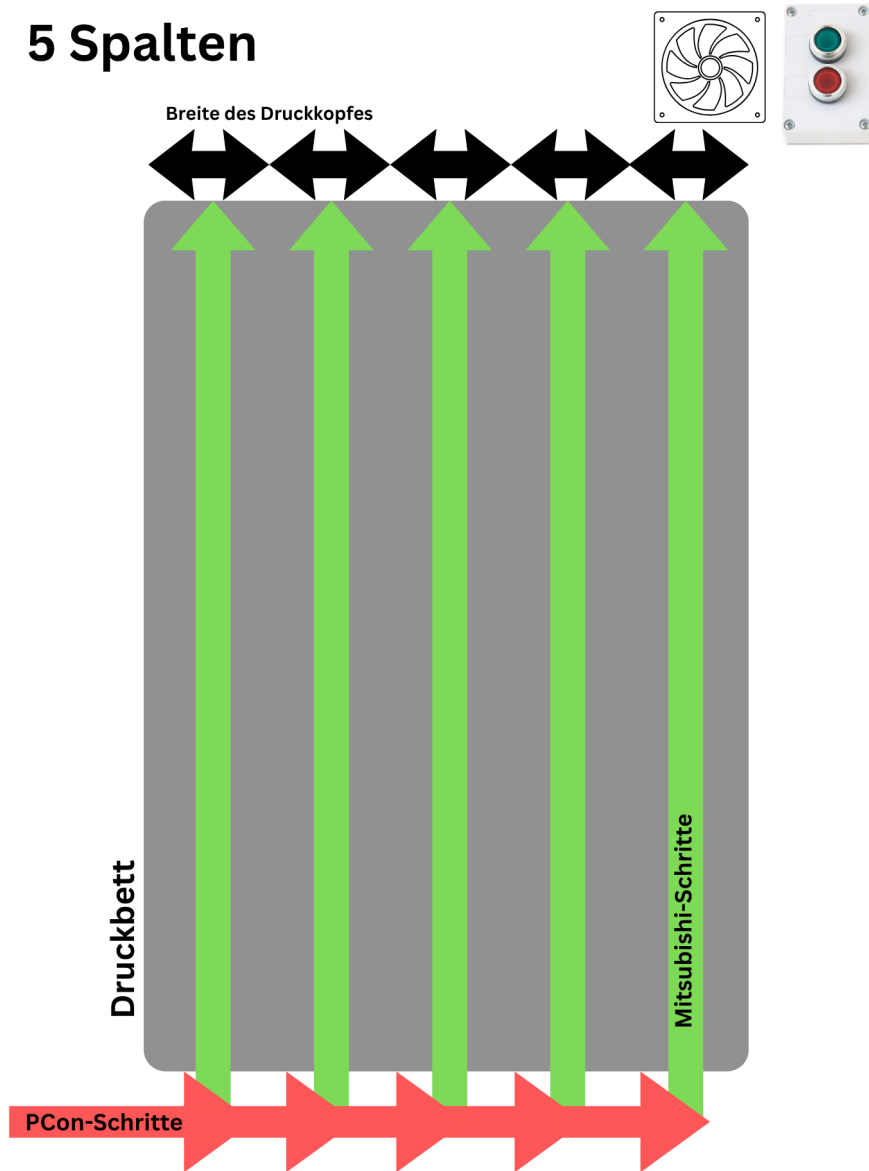


Abbildung 7: Ablauf bei 5 Spalten

Quellenverzeichnis

- [1] International Electrotechnical Commission. *IEC 61131-1: Programmable Controllers – Part 1: General Information*. International Electrotechnical Commission, 2003. URL: <https://webstore.iec.ch/en/publication/4551> (besucht am 21. 09. 2026).
- [2] Siemens AG. *SIMATIC: Automatisierung mit speicherprogrammierbaren Steuerungen*. Informationen des Herstellers zur industriellen Automatisierung mit SIMATIC-Steuerungen. 2026. URL: <https://www.siemens.com/global/en/products/automation/systems/industrial/plc.html> (besucht am 21. 09. 2026).
- [3] AGG Software. *Advanced USB Port Monitor*. Offizielle Produktinformationen des Herstellers. 2026. URL: <https://www.usb-port-monitor.com/> (besucht am 21. 09. 2026).
- [4] Python Software Foundation. *Python 3 Documentation*. Offizielle Python-Dokumentation. 2026. URL: <https://docs.python.org/3/> (besucht am 21. 09. 2026).
- [5] Python Software Foundation. *The Python Standard Library: time — Time access and conversions*. Offizielle Dokumentation des Python-Moduls time. 2026. URL: <https://docs.python.org/3/library/time.html> (besucht am 21. 09. 2026).
- [6] Python Software Foundation. *The Python Standard Library: sys — System-specific parameters and functions*. Offizielle Dokumentation des Python-Moduls sys. 2026. URL: <https://docs.python.org/3/library/sys.html> (besucht am 21. 09. 2026).
- [7] Al Sweigart. *PyAutoGUI Documentation*. Dokumentation der Python-Bibliothek PyAutoGUI. 2026. URL: <https://pyautogui.readthedocs.io/> (besucht am 21. 09. 2026).
- [8] Charles R. Harris u. a. „Array programming with NumPy“. In: *Nature* 585 (2020), S. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: <https://doi.org/10.1038/s41586-020-2649-2>.
- [9] Aivar Annamaa. *Thonny: Python IDE for Beginners*. Offizielle Website der Entwicklungsumgebung Thonny. 2026. URL: <https://thonny.org/> (besucht am 21. 09. 2026).

Eidesstattliche Erklärung

Ich erkläre hiermit, dass ich diese Arbeit selbstständig ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Alle den benutzten Quellen wörtlich oder sinngemäß entnommenen Stellen sind als solche einzeln kenntlich gemacht.

Diese Arbeit ist bislang keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht worden.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Leipzig, 22. September 2026

.....
Jonathan Blänsdorf

.....
Paul Felix Freiburger

Hilfsmittelverzeichnis

Im Rahmen der vorliegenden Bachelorarbeit wurden die folgenden Hilfsmittel verwendet:

- **Textverarbeitung und Satz:** Overleaf und LaTeX.
- **Künstliche Intelligenz:** SiemensGPT wurde zur Strukturierung und Unterstützung bei der Erstellung von LaTeX-Code und zur Recherche technischer Dokumentationen eingesetzt. Die fachliche Prüfung, Auswahl und Überarbeitung aller Inhalte erfolgte eigenständig.